FIT2109 Computer Science Workshop

Week 12: Agentic Coding & AI Tools

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

The last skill: working with AI

> *"The question is not whether AI will write code. It already does. The question is whether you understand what it wrote."*

Weeks 1–11 built the professional toolkit: shell, Git, containers, debugging, profiling. Week 12 asks how AI systems fit into that toolkit — and where they break down. The goal is to use GenAI tools and coding agents *effectively and safely*, not uncritically.

The central message

AI is a helper, not a replacement for engineering skill. It is useful only when you are strong enough to review, test, debug, and explain what it produced.

Start with concepts, then products

Stable concepts

GenAI Models that generate text, code, or artefacts from prompts.

GenAI API A way to call a model from your software.

Chatbot A conversational interface: ask, answer, copy, paste.

Agent A system that uses tools, observes results, and continues toward a goal.

CLI agent A terminal agent close to shell, Git, builds, and tests.

Concrete examples

- ChatGPT, Claude, Gemini: chat interfaces.
- Codex CLI, Claude Code, Gemini CLI: terminal coding agents.
- Cline, Aider, Copilot: IDE or Git-centred tools.
- OpenClaw: autonomous-agent framework.

Principle: concepts first; products second. Tool names change faster than workflow habits.

Learning goals

By the end of this workshop, you should be able to:

- Distinguish GenAI models, chatbots, coding agents, and CLI agents.
- Use a GenAI API for narrow software tasks such as classification, extraction, and summarisation.
- Explain what AI agents can and cannot do reliably.
- Write effective prompts and project instruction files, such as `CLAUDE.md` or `AGENTS.md`, that produce useful output.
- Use a CLI coding agent, such as Codex CLI or Claude Code, to scaffold, explain, and refactor code.
- Explain what MCP is and what a **harness** does.
- Verify AI-generated output critically before accepting it.

Concept 1: Three shifts made GenAI practical

Three things changed at once

Architecture **Transformers** (2017) — attention over long contexts replaced older sequence models.

Scale Models trained on essentially all public code on the internet.

APIs Capability packaged as a service — no research lab required to use it.

Why code specifically?

Code is the ideal training target:

- Vast public supply (GitHub).
- Highly structured and consistent.
- Correctness is measurable — tests pass or fail.
- Huge developer demand willing to pay for it.

Concept 2: GenAI APIs put models inside software

When an API is the right tool

- Classify a support message or bug report.
- Extract structured fields from messy text.
- Summarise logs, reviews, or meeting notes.
- Draft text that a human will edit.

When not to use it

Do not use a model API where deterministic code is clearer, cheaper, safer, or more testable. Sorting a list, parsing a fixed format, and checking a rule should stay normal code.

API pattern

1. Define the task.
2. Provide only necessary context.
3. Ask for structured output.
4. Validate the result in code.
5. Handle errors, cost, and privacy.

Demo: a GenAI API as a classifier

Watch

Treat the model as one component inside a normal program: define labels, request structured output, parse it, and validate it.

Note

The engineering work is the boundary around the model: schema, checks, tests, errors, cost, and privacy.

Concept 3: AI is fast at patterns, weak at guarantees

Strong at

- Boilerplate and scaffolding.
- Explaining existing code.
- Generating unit tests from a spec.
- Completing familiar patterns quickly.

Reliably fails at

- Guaranteeing correctness.
- Novel algorithms or proofs.
- Knowing your specific codebase.
- Security awareness by default.

Key insight

AI predicts *likely* text, not *correct* logic. It can hallucinate API names, flags, and signatures with full confidence.

If you cannot write, trace, test, and debug the code yourself, you cannot judge whether the AI helped or harmed you.

Concept 4: Agents can act, not just answer

A chatbot

- You ask, it answers in text.
- One turn at a time.
- Basic chat sees only supplied context.
- You paste changes manually.

An agent

- Given a **goal**, not just a prompt.
- Reads files, runs commands, inspects output.
- Loops: plan → act → observe → revise.
- Stops when the goal is met — or when stuck.

The read → plan → act loop

Read context (files, errors, history)

↓

Plan next action (which tool to call)

↓

Act and observe (run, read result)

↓ *repeat until done*

Tool names are examples

Chat: **ChatGPT**, **Claude**, **Gemini**.
Terminal agents: **Codex CLI**, **Claude Code**, **Gemini CLI**.
Editor/Git tools: **Cline**, **Aider**, **Copilot**.
General agents: **OpenClaw**.

Handout Activity 1: model, API, chatbot, agent

Now work on this activity in the handout

Classify AI-tool scenarios by what the tool can see, what it can do, and where code must still validate the result.

Group output

Complete the comparison table and one API-boundary design.

A typical CLI agent session

```
cd my-project
git status

codex --sandbox workspace-write \
  --ask-for-approval on-request \
  "Inspect this repo and suggest one
  safe first improvement."

git diff
make test
```

What is happening?

1. Start inside a Git repo.
2. Give a narrow goal.
3. Let the agent read, edit, and run commands.
4. Review the diff yourself.
5. Run tests before accepting.

The CLI starts the conversation; Git and tests decide whether the result is acceptable.

Codex CLI at a glance

Interactive work

`codex` Open the terminal UI.

`codex "prompt"` Start with a prompt.

`codex -cd DIR` Start in another repo.

`codex resume` Continue a session.

Automation and checks

`codex exec` Run non-interactively.

`codex review` Review a diff.

`codex doctor` Diagnose setup.

`AGENTS.md` Project instructions.

Default: least access

Use the narrowest permission needed. Avoid full access unless you are in a controlled, disposable environment.

Concept 5: Better prompts give better constraints

The three ingredients

Task What you want done. Specific, not vague.

Context Language, framework, existing structure.

Constraints What it must *not* do: don't modify tests, no new dependencies.

Treat prompts like code

Vague input → vague output. If the result is wrong, improve the prompt first.

Vague vs specific

Vague: "Fix my code."

Specific: "`find_matches()` in `data.py` is $O(n^2)$. Rewrite using a set. Do not change the signature or tests."

Demo 1: prompts that work vs fail

Watch

Compare a vague prompt with a scoped prompt on the same tiny bug, then inspect the proposed diff.

Note

A good prompt creates a small reviewable change; it does not remove your responsibility to review.

Concept 6: Project context keeps agents grounded

What it is

A Markdown file at the project root. Claude Code reads `CLAUDE.md`; Codex uses `AGENTS.md`. Different filenames, same purpose: ground the agent in your project reality.

What to put in it

- Project purpose and architecture overview.
- Commands to build, test, and lint.
- Naming conventions and style rules.
- Files the agent should not touch.
- Known constraints (no new pip dependencies).

```
# MyProject

## Commands
- `make test`
- `make lint`

## Rules
- snake_case for functions
- stdlib only; no new deps

## Do not touch
- legacy/migration.py
```

Demo 2: `CLAUDE.md` in action

Watch

Add project context rules, ask for a small change, and observe whether the agent follows the local constraints.

Note

Project context is not magic. It is useful only when it is short, accurate, and paired with verification.

Handout Activity 2: prompts and project context

Now work on this activity in the handout

Rewrite vague agent tasks into scoped prompts and decide what belongs in a project context file.

Group output

Complete the prompt rewrite, context-file checklist, and no-touch rule.

Concept 7: MCP connects agents to external tools

What MCP is

Model Context Protocol — an open standard introduced by Anthropic (2024), now community-governed, that lets AI agents connect to external tools and data sources in a consistent way.

What MCP servers expose

Tools Functions the agent can call: run a query, create a PR, send a message.

Resources Data the agent can read: files, docs, database rows.

Prompts Pre-built instruction templates.

MCP servers in practice

- Filesystem — read/write local files.
- GitHub — list issues, create PRs.
- PostgreSQL — query a database.
- Slack — send and read messages.
- Browser — navigate and scrape pages.

Why it matters

The agent's power comes from its tools, not just its model weights. MCP is how tools are added safely and consistently.

Concept 8: Agents need a control layer

What a **harness** is

The infrastructure layer that controls what an agent can do — before it does it.

Four components

Context What the agent knows (`CLAUDE.md`, system prompt).

Allowlist Which tools it may call (`-allowedTools`).

Approval gate Human confirmation for destructive actions.

Audit log A record of every action taken.

Claude Code's built-in harness

- `CLAUDE.md` → context layer.
- `-allowedTools` → tool allowlist.
- Permission prompts → approval gate.
- Session transcript → audit log.

Without a harness

A powerful agent given an ambiguous goal will do *something* — just not necessarily what you intended.

Concept 9: Agent mistakes become real actions

Why security matters for agents

- An agent with shell access can delete files, exfiltrate data, or install packages.
- It does what the task requires — even if that is destructive.
- Agents can enter loops: repeated tool calls that make no progress but cause side effects.
- **Prompt injection**: malicious content in a file the agent reads can hijack its instructions.

Practical rules

1. Always review the proposed diff before accepting.
2. Never paste credentials into a prompt.
3. Use `-allowedTools` to limit scope.
4. Run agents in a container for untrusted tasks.
5. Treat agent-written code as a code review, not a gift.

Handout Activity 3: MCP, harnesses, and boundaries

Now work on this activity in the handout

Design a narrow tool boundary and identify where prompt injection or over-permission can cause harm.

Group output

Complete the MCP permission table and one safety-harness sketch.

Concept 10: Verification is part of using AI

The verification workflow

1. **Read the diff** — every line, before accepting.
2. **Run the tests** — passing tests increase confidence; failing tests must be investigated.
3. **Explain it** — can you describe what each changed line does?
4. **Check edge cases** — empty input, large input, unexpected types.

Academic integrity

You may use AI tools **if** you disclose it and can explain every line you submit.
In this unit, you should expect to explain code you submit: "Walk me through what this code does."
Submitting code you cannot explain is academic misconduct — regardless of who, or what, wrote it.

Demo 4: agent-generated code still needs review

Watch

Let an agent make a plausible change, then inspect the diff for scope creep, hidden assumptions, and missing tests.

Note

The danger is not only wrong code; it is confident code that looks finished before it has been verified.

Demo 4 continued: verify before trusting

Watch

Run the existing checks, add or run a targeted test, and compare the evidence with the original task.

Note

AI assistance ends with a human-quality verification story, not with generated text.

Handout Activity 4: verification before trust

Now work on this activity in the handout

Write a verification plan for an agent-produced change before deciding whether to accept it.

Group output

Complete the diff-review checklist, test evidence plan, and final accept/reject decision.

Final checkpoint: Poll Everywhere

pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to `pollev.com/fit2109`.

Purpose

Use the Poll Everywhere questions to check which Week 12 ideas need one more example before we finish.

What should feel different after today?

Before → After

Magic / hype	→ Understood read → plan → act loop
Vague prompts	→ Task + context + constraints
Trust blindly	→ Review every diff
Chatbot mindset	→ MCP, harness, verify
AI dependence	→ Strong coder using AI selectively

The unit in one picture

Shell + Git + Containers + Debugging + Profiling + **Agents**

Each week added a layer. Agents are most useful to someone who understands the layers underneath.

The rule

An AI tool is powerful in the hands of someone who knows the fundamentals. It is a liability otherwise. Do not rely on AI for everything; use it where you can still own the result.

Q&A and end of semester

Today we covered

- Why GenAI is popular now — transformers, scale, code data.
- GenAI APIs for narrow tasks: classify, extract, summarise, validate.
- Capabilities and limitations — prediction vs. correctness.
- Agents vs chatbots; CLI-agent usage with Codex, Claude Code, and Gemini CLI.
- Prompting, project context files, MCP, harnesses, and security.
- Verifying output, academic integrity, and why coding still matters.

End of semester

This was the last seminar. Good luck with your group project — due **Friday Week 14**. Post questions on Ed; office hours continue.